

Analysis Tools

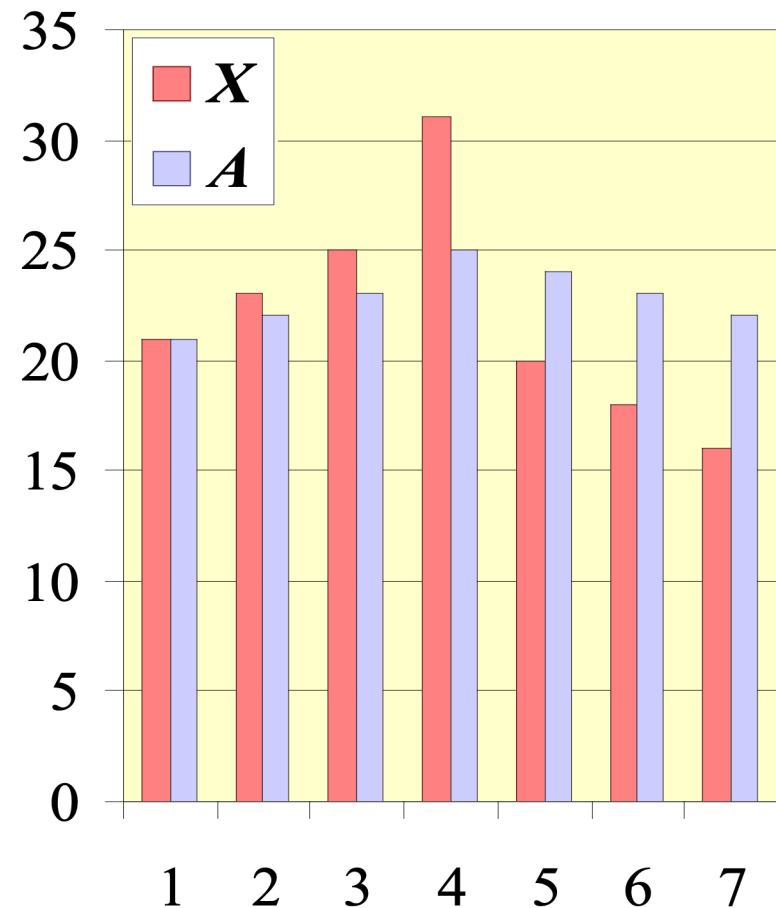
- Counting Primitive Operations
- Asymptotic Analysis
 - and **examples**

Examples

- Computing Prefix Averages
 - Version 1: Quadratic time
 - Version 2: Linear time
- Computing powers recursively
- Computing Fibonacci numbers

Example: Computing Prefix Averages

- We illustrate asymptotic analysis with two algorithms for prefix averages
- The i -th prefix average of an array X is average of the first $(i + 1)$ elements of X
$$A[i] = (X[0] + X[1] + \dots + X[i]) / (i + 1)$$
- Computing the array A of prefix averages of another array X has applications to financial analysis and statistics



Prefix Averages (quadratic time)

Algorithm *prefixAverages1*(X, n)

Input array X of n integers

Output array A of prefix averages of X

$A \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do**

$s \leftarrow 0$

for $j \leftarrow 0$ **to** i **do**

$s \leftarrow s + X[j]$

 { increment j }

$A[i] \leftarrow s \div (i + 1)$

 { increment i }

return A

Always document input/output

#operations

$1 + n$

$1 + 2(n+1)$

n

?

$4n$

n

1

Prefix Averages (quadratic time)

Algorithm *prefixAverages1*(X, n)

Input array X of n integers

Output array A of prefix averages of X #operations

$A \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do** $i = 0$

$s \leftarrow 0$

for $j \leftarrow 0$ **to** i **do** $1 + 1 + 1$

$s \leftarrow s + X[j]$ 3

 { increment j } 1

$A[i] \leftarrow s \div (i + 1)$

 { increment i }

return A

Prefix Averages (quadratic time)

Algorithm *prefixAverages1*(X, n)

Input array X of n integers

Output array A of prefix averages of X #operations

$A \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do** $i = 1$

$s \leftarrow 0$

for $j \leftarrow 0$ **to** i **do** $1 + 2 + 1$

$s \leftarrow s + X[j]$ 6

 { increment j } 2

$A[i] \leftarrow s \div (i + 1)$

 { increment i }

return A

Prefix Averages (quadratic time)

Algorithm *prefixAverages1*(X, n)

Input array X of n integers

Output array A of prefix averages of X

#operations

$A \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do**

$i = 0, 1, \dots, n-1$

$s \leftarrow 0$

for $j \leftarrow 0$ **to** i **do**

$1 + (i + 1) + 1$

$s \leftarrow s + X[j]$

$3(i + 1)$

 { increment j }

$i + 1$

$A[i] \leftarrow s \div (i + 1)$

 { increment i }

return A

Prefix Averages (quadratic time)

Algorithm *prefixAverages1*(X, n)

Input array X of n integers

Output array A of prefix averages of X

$A \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do**

$s \leftarrow 0$

for $j \leftarrow 0$ **to** i **do**

$s \leftarrow s + X[j]$

 { increment j }

$A[i] \leftarrow s \div (i + 1)$

 { increment i }

return A

#operations

$1 + n$

$1 + 2(n+1)$

n

$n + (1 + 2 + \dots + n) + n$

$3 \cdot (1 + 2 + \dots + n)$

$1 + 2 + \dots + n$

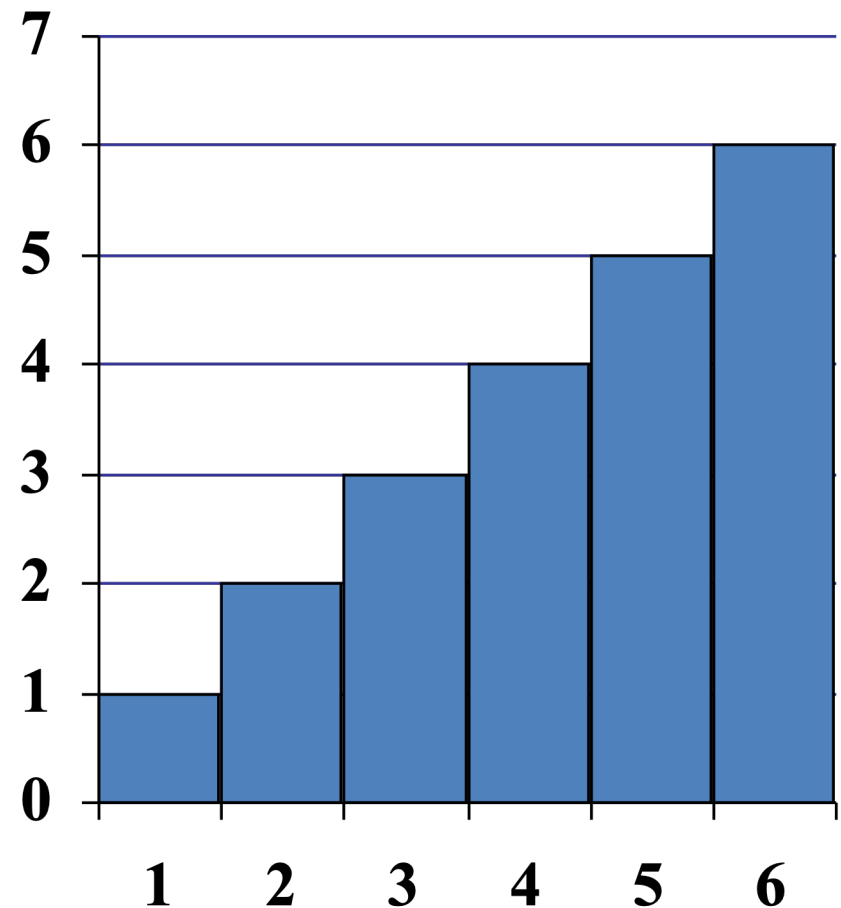
$4n$

n

1

Algorithm Dominated by a “Geometric Sum”

- Running time of *prefixAverages1* is order $1 + 2 + \dots + n$
- The sum of the first n integers is $n(n + 1) \div 2$
 - simple visual proof $\rightarrow$
- Thus, *prefixAverages1* runs in $O(n^2)$ time



Prefix Averages (linear time)

Algorithm *prefixAverages2*(X, n)

Input array X of n integers

Output array A of prefix averages of X

$A \leftarrow$ new array of n integers

$s \leftarrow 0$

for $i \leftarrow 0$ **to** $n - 1$ **do**

$s \leftarrow s + X[i]$

 See what we did here?

$A[i] \leftarrow s \div (i + 1)$

 { increment i }

return A

#operations

n

1

$1 + 2(n + 1)$

$3n$

$4n$

n

1

$11n + 5$ is $O(n)$

Example: Computing Powers (recursively)

$$p(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot p(x,n-1) & \text{otherwise} \end{cases}$$

$$2^4 = 2(2^3) = 2(2(2^2)) = 2(2(2(2^1))) = 2(2(2(2(1)))) = 16$$

$$2^5 = 2(2^4) = 2(2(2^3)) = 2(2(2(2^2))) = 2(2(2(2(2^1)))) = 2(2(2(2(2(1))))) = 32$$

$$2^6 = 2(2^5) = 2(2(2^4)) = 2(2(2(2^3))) = 2(2(2(2(2^2)))) = 2(2(2(2(2(2^1))))) = 2(2(2(2(2(2(1))))) = 64$$

$$2^7 = 2(2^6) = \dots = 128$$

Computing Powers (naive algorithm)

Algorithm *power*(x, n)

Input A number x and integer $n \geq 0$

Output The value x^n **#operations**

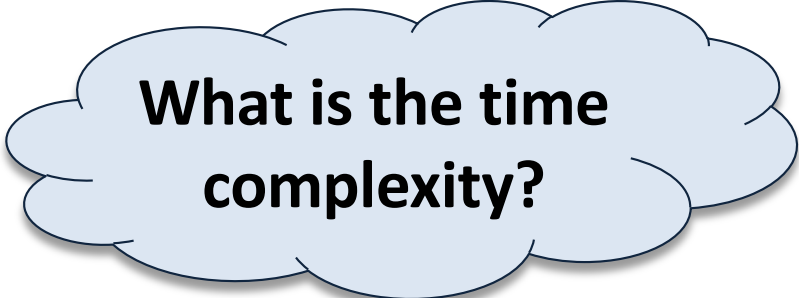
if $n = 0$ **then** **1**

return 1 **1**

else

$y \leftarrow x \cdot \text{power}(x, n - 1)$ **4**

return y **1**



What is the time complexity?

Computing Powers (naive algorithm)

Algorithm *power*(x, n)

Input A number x and integer $n \geq 0$

Output The value x^n

if $n = 0$ **then**

return 1

else

$y \leftarrow x \cdot \text{power}(x, n - 1)$

return y

#operations

1

1

4

1

Each time we make a recursive call we will decrease n by 1; hence we make n recursive calls

Number of steps: $2+6n$ is $O(n)$ so LINEAR

note: we count the test for the else case too
hence we have 6 operations

Example: Computing Powers (by squaring)

$$p(x, n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot p(x, (n-1)/2)^2 & \text{if } n \text{ is odd} \\ p(x, n/2)^2 & \text{if } n \text{ is even} \end{cases}$$

$$2^4 = 2^{(4/2)^2} = (2^{4/2})^2 = (2^2)^2 = 4^2 = 16$$

$$2^5 = 2^{1+(4/2)^2} = 2(2^{4/2})^2 = 2(2^2)^2 = 2(4^2) = 32$$

$$2^6 = 2^{(6/2)^2} = (2^{6/2})^2 = (2^3)^2 = 8^2 = 64$$

$$2^7 = 2^{1+(6/2)^2} = 2(2^{6/2})^2 = 2(2^3)^2 = 2(8^2) = 128$$

Example: Computing Powers (by squaring)

$$p(x, n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot p(x, (n-1)/2)^2 & \text{if } n \text{ is odd} \\ p(x, n/2)^2 & \text{if } n \text{ is even} \end{cases}$$

operations

$$2^9 = 2^5 \times 2^4$$

$$2^4 = 2^2 \times 2^2 \quad 1 \text{ (} 2^2 \text{ already computed)}$$

$$2^2 = 2 \times 2 \quad 1$$

$$2^5 = 2^3 \times 2^2 \quad 1 \text{ (both } 2^3 \text{ and } 2^2 \text{ already computed)}$$

$$2^3 = 2^2 \times 2 \quad 1 \text{ (} 2^2 \text{ already computed)}$$

5 operations total.

$$\log_2(9) = 3.17 \text{ (rounds to 4)}$$

$$\log_2(25) = 4.64 \text{ (rounds to 5)}$$

$$\log_2(50) = 5.64 \text{ (rounds to 6)}$$

$$\log_2(255) = 7.99 \text{ (rounds to 8)}$$

...

For large power N , the number of ops is $\log_2(N)$

Example: Computing Powers (by squaring)

Algorithm *power2*(x, n)

Input: A number x and integer $n \geq 0$

Output: The value x^n

if $n = 0$ **then**

return 1

if n is odd **then**

$y \leftarrow \text{power2}(x, (n - 1) \div 2)$

return $x \cdot y \cdot y$

else

$y \leftarrow \text{power2}(x, n \div 2)$

return $y \cdot y$

Each time we make a recursive call we halve the value of n , hence, we make $\log_2 n$ recursive calls. That is, this method runs in $O(\log_2 n)$ time.

It is prudent to use a variable y here rather than calling the method twice. (Can you say why?)

Computing Fibonacci Numbers

- Fibonacci numbers are defined recursively

$$F_0 = 0$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2} \quad \text{for } i > 1$$

- Recursive algorithm (naïve attempt)

Algorithm *BinaryFib*(k):

Input: Non-negative integer k

Output: The k^{th} Fibonacci number F_k

if $k < 2$ **then**

return k

else

return *BinaryFib*($k - 1$) + *BinaryFib*($k - 2$)

Fibonacci sequence

- Let n_k be the number of calls to **BinaryFib**(k)

- $n_0 = 1$
- $n_1 = 1$
- $n_2 = n_1 + n_0 = 1 + 1 = 2$
- $n_3 = n_2 + n_1 = 2 + 1 = 3$
- $n_4 = n_3 + n_2 = 3 + 2 = 5$
- $n_5 = n_4 + n_3 = 5 + 3 = 8$
- $n_6 = n_5 + n_4 = 8 + 5 = 13$
- $n_7 = n_6 + n_5 = 13 + 8 = 21$
- $n_8 = n_7 + n_6 = 21 + 13 = 44$

Algorithm *BinaryFib*(k):

Input: Non-negative integer k

Output: The k^{th} Fibonacci number F_k

if $k < 2$ **then**

return k

else

return *BinaryFib*($k - 1$) + *BinaryFib*($k - 2$)

Analysis (counting recursive calls)

- Let c_k be the number of calls to **BinaryFib**(k)

- $c_0 = 1$
- $c_1 = 1$
- $c_2 = c_1 + c_0 + 1 = 1 + 1 + 1 = 3$
- $c_3 = c_2 + c_1 + 1 = 3 + 1 + 1 = 5$
- $c_4 = c_3 + c_2 + 1 = 5 + 3 + 1 = 9$
- $c_5 = c_4 + c_3 + 1 = 9 + 5 + 1 = 15$
- $c_6 = c_5 + c_4 + 1 = 15 + 9 + 1 = 25$
- $c_7 = c_6 + c_5 + 1 = 25 + 15 + 1 = 41$
- $c_8 = c_7 + c_6 + 1 = 41 + 25 + 1 = 67$

Algorithm *BinaryFib*(k):

Input: Non-negative integer k

Output: The k^{th} Fibonacci number F_k

if $k < 2$ **then**

return k

else

return *BinaryFib*($k - 1$) + *BinaryFib*($k - 2$)



overlapping

- Note that c_k at least doubles every other time
- That is, $c_k > 2^{k/2}$. **It is exponential!**

Better Fibonacci Algorithm?

Algorithm *BetterFib*(k):

Input: Non-negative integer k

Output: Pair of Fibonacci numbers (F_k, F_{k-1})

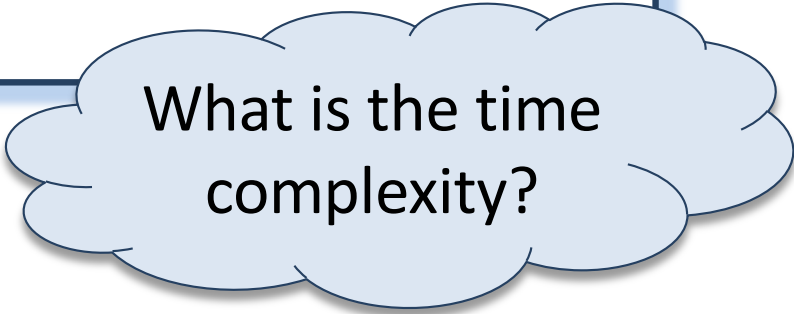
if $k = 1$ **then**

return $(k, 0)$

else

$(i, j) \leftarrow \text{BetterFib}(k - 1)$

return $(i + j, i)$



What is the time complexity?

Better Fibonacci Algorithm?

Example Execution:

For $k = 4$, the function would proceed as follows:

1. `BetterFib(4)` calls `BetterFib(3)`:
 - `BetterFib(3)` calls `BetterFib(2)`:
 - `BetterFib(2)` calls `BetterFib(1)`, which returns $(1, 0)$.
 - `BetterFib(2)` then returns $(1 + 0, 1) = (1, 1)$.
 - `BetterFib(3)` then returns $(1 + 1, 1) = (2, 1)$.
2. Finally, `BetterFib(4)` returns $(2 + 1, 2) = (3, 2)$.

So, $F_4 = 3$ and $F_3 = 2$.

Better Fibonacci Algorithm?

Complexity:

- **Time Complexity:**
 - The time complexity is $O(k)$ because each call to `BetterFib(k)` results in a single call to `BetterFib(k-1)`. This leads to a linear recursion depth of k , with each level of recursion performing constant work (adding two integers).
- **Space Complexity:**
 - The space complexity is also $O(k)$, primarily due to the recursion stack. Each recursive call adds a new frame to the stack, and since the maximum depth of recursion is k , the space complexity corresponds to this depth.

Fibonacci iterative (loop) approach

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    int a = 0, b = 1;  
    for (int i = 2; i <= n; i++) {  
        int next = a + b;  
        a = b; // memorisation  
        b = next;  
    }  
    return b;  
}
```

$O(n)$ time complexity

Fibonacci Recursive Power Matrix

The Key Idea

The Fibonacci sequence can be expressed with the following recurrence relation:

$$F(n) = F(n - 1) + F(n - 2)$$

This relation can be represented in matrix form as:

$$\begin{pmatrix} F(n) \\ F(n - 1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F(n - 1) \\ F(n - 2) \end{pmatrix}$$

This matrix equation indicates that the Fibonacci sequence can be generated by multiplying the matrix:



Fibonacci Recursive Power Matrix

$$\begin{pmatrix} F(n) \\ F(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F(n-1) \\ F(n-2) \end{pmatrix}$$

Matrix Representation of Fibonacci Sequence

Given the above, you can represent the computation of Fibonacci numbers as follows:

$$\begin{pmatrix} F(n) \\ F(n-1) \end{pmatrix} = A^{(n-1)} \begin{pmatrix} F(1) \\ F(0) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{(n-1)} \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

So, to compute $F(n)$, you can exponentiate the matrix A to the power of $n - 1$ and multiply it by the initial vector $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$.

Fibonacci Recursive Power Matrix

Matrix Exponentiation

Matrix exponentiation is the process of raising a matrix to a power. While a naive approach would require $O(n)$ multiplications, we can do better using a technique called **exponentiation by squaring**, which allows us to compute A^n in $O(\log n)$ time. This is how it works:

1. Base Case:

- $A^0 = I$ (identity matrix)
- $A^1 = A$

2. Recursion:

- If n is even, $A^n = (A^{n/2}) \times (A^{n/2})$
- If n is odd, $A^n = A \times (A^{(n-1)/2}) \times (A^{(n-1)/2})$

This method reduces the time complexity of computing Fibonacci numbers to $O(\log n)$, which is a significant improvement over the $O(n)$ time complexity of iterative or naive recursive approaches.

Java implementation

Recap of Analysis

- Counting primitive operations
 - Determine $f(n)$ – where n is size of input
- Measuring asymptotic time complexity
 - Only concerned with growth rate
 - drop constant factors and lower order terms
 - Seven important classes of functions
 - within-class differences non-essential
 - between-class differences significant “in the large”
 - $f(n)$ can be expressed using Big-O notation
 - recall Big-Omega and Big-Theta...
 - allows “bounds” on asymptotic behaviour to be formalised
 - provides common notation for describing time and space complexity of algorithms